

Chapter 19: Reasoning Models, Test-Time Compute, and GRPO

Preceding Bridge: In [Chapter 01: LLM Foundations](#) and [Chapter 14: Model Fine-Tuning & DPO](#), you learned standard next-token autoregressive generation and preference alignment. In this chapter, we explore the definitive AI paradigm shift of 2025/2026: **Test-Time Compute, Thinking Tokens (<think> ... </think>)**, and **Group Relative Policy Optimization (GRPO)** popularized by DeepSeek-R1 and OpenAI o1/o3.

1. Plain-English Jargon Demystifier

Technical Term	Plain English Translation	Real-World Metaphor
Test-Time Compute	Allowing the model to generate internal reasoning tokens before outputting its final response.	A student writing calculations on scratch paper before writing the final answer on the exam sheet.
Thinking Tokens (<think>)	Hidden internal tokens where the model explores hypotheses, backtracks, and checks its own logic.	The internal monologue inside your head before you speak aloud.
PPO (Proximal Policy Optimization)	Classic RL method requiring a heavy second neural network (Critic) to estimate state values.	A student taking a test with an expensive private tutor standing over their shoulder scoring every move.
GRPO (Group Relative Policy Optimization)	Modern RL that samples a group of answers to the same question and scores them relative to the group average (No Critic!).	A teacher having 8 students solve the same problem on a whiteboard and grading each relative to the class mean.
RLVR (RL with Verifiable Rewards)	Training models on tasks with unambiguous ground-truth answers (math, code execution, formal logic).	A compiler or calculator automatically giving a green checkmark or red X. Zero human labelers needed!

2. Spoon-Fed Mental Model: Blitz Chess vs Grandmaster Deep

Thought

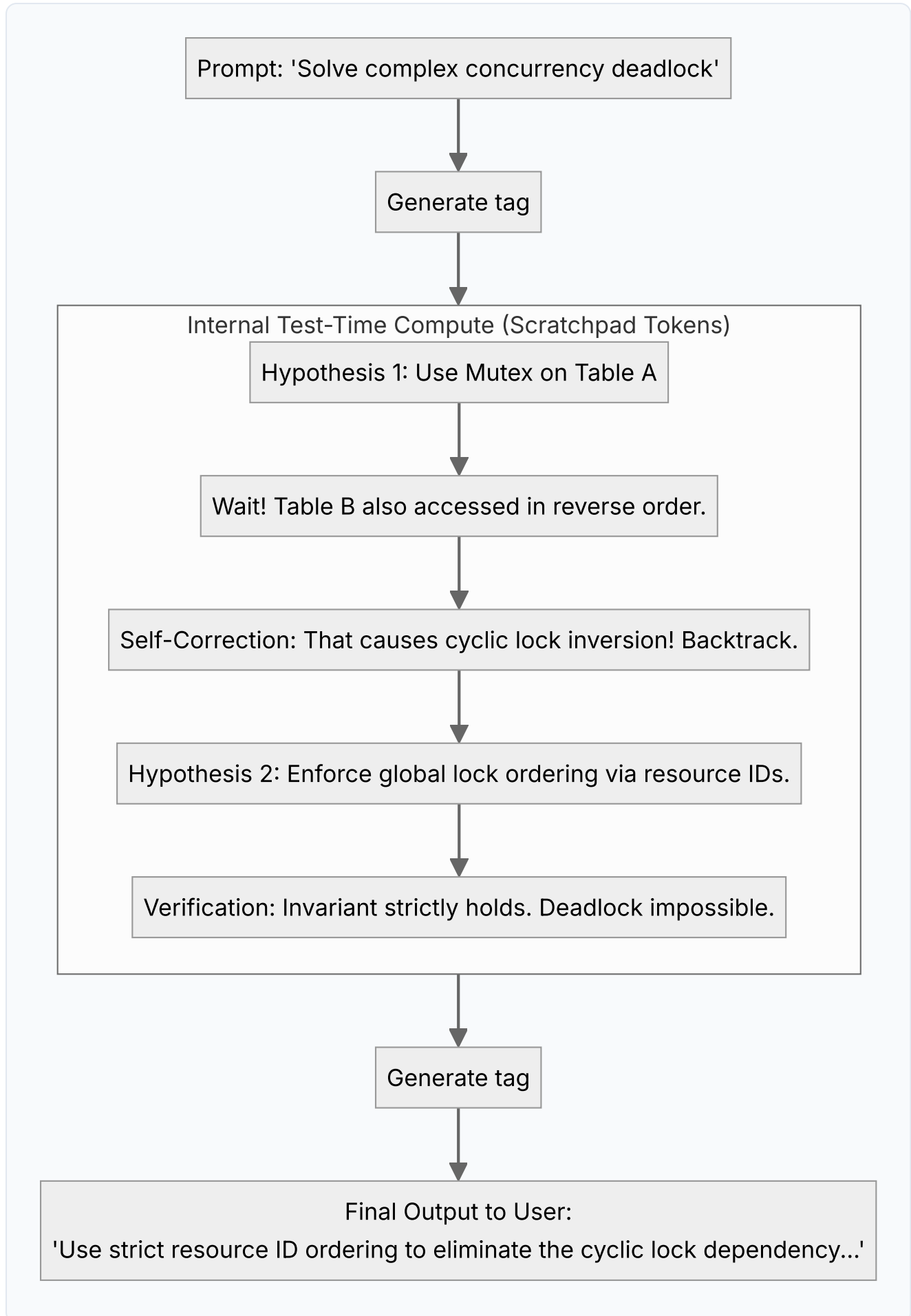
Imagine two chess players:

Pre-2025 LLMs (Next-Token Prediction):

- Plays **Speed Blitz Chess**: Must move a piece every **0.5 seconds**.
- If you ask a 10-step math problem or a complex security audit question, it outputs the very first word that sounds statistically likely.
- If it takes a wrong turn at Step 3, **it cannot go back!** It stubbornly doubles down on the mistake and hallucinates the rest of the answer.

2025/2026 Reasoning Models (Test-Time Compute):

- Plays **Classical Tournament Chess**: Has **20 minutes of clock time** to deliberate.
- It explores Branch A: *"Wait, if I move the rook here, black counters with knight. That's a trap!"*
- It **backtracks** and explores Branch B: *"What if I sacrifice the bishop first? Yes, that forces checkmate in 4."*
- Only after verifying its logic on internal scratchpad tokens does it produce the final move.



3. The Mathematics of GRPO (Group Relative Policy Optimization)

In standard RLHF (PPO), training requires **two large models loaded simultaneously into GPU VRAM**:

1. The **Actor Model** (π_θ): Generates the text.
2. The **Critic Model** (V_ϕ): Estimates the expected reward of every token. *Problem*: The Critic takes as much GPU memory as the Actor, halving your effective batch size!

How DeepSeek-R1 Eliminates the Critic with GRPO:

For each input prompt q , the model samples a **group of G candidate outputs** $\{o_1, o_2, \dots, o_G\}$. Each output o_i receives a verifiable scalar reward r_i (e.g., 1.0 if unit tests pass, 0.0 if they fail).

The **Advantage** $\hat{A}_i$ is computed simply by standardizing rewards against the group mean and standard deviation:

$$\hat{A}_i = \frac{r_i - \text{mean}(\{r_1, \dots, r_G\})}{\text{std}(\{r_1, \dots, r_G\})}$$

Why GRPO is revolutionary: It eliminates the Critic network completely, freeing up **50% of GPU memory** and allowing massive reinforcement learning runs on consumer-accessible clusters!

4. Junior vs Staff Implementation: Integrating Reasoning Models

5. Complete Runnable Implementation: GRPO Advantage & Reasoning Parser

Here is a 100% runnable, zero-dependency Python script demonstrating GRPO group advantage calculation and a production reasoning token streaming parser:

6. Chapter Milestone Check

Verify your understanding before continuing:

1. How does Group Relative Policy Optimization (GRPO) eliminate the need for a Critic network?

- *Answer:* Instead of training a separate multi-billion-parameter neural network to predict the reward baseline of each state, GRPO samples a cohort of G responses for the same query and scores each response relative to the cohort's empirical mean and standard deviation.

2. What is Test-Time Compute and why does it outperform pure pre-training scaling for reasoning tasks?

- *Answer:* Pre-training scaling only improves the model's static statistical intuition. Test-Time Compute grants the model dynamic inference-time token budget to explore hypotheses, verify invariants, detect contradictions, and self-correct before outputting the final token.

3. Why must autonomous agent architectures separate thinking tokens from final response streams?

- *Answer:* Leaking raw chain-of-thought scratchpad tokens clutters user interfaces, increases downstream token usage when fed back into chat history, and may expose internal prompt instructions.

Further Reading

- [DeepSeekMath \(GRPO\)](#)
 - [DeepSeek-R1 paper](#)
 - [Chain-of-thought prompting paper](#)
-

Check Yourself

Answer in your head or on paper first, then open each answer.

▼ 1. What is test-time compute?

Spending more computation at inference (longer reasoning, sampling several answers, search) to improve answers.

▼ 2. How does GRPO avoid a value model?

It samples a group of answers per prompt and uses each answer's reward relative to the group mean as the advantage.

▼ 3. What makes a reward 'verifiable'?

It can be checked automatically (unit tests, exact answers), enabling RL without human labels.

▼ 4. Why can longer reasoning hurt cost and latency?

More tokens are generated and billed; use budgets and routing to reasoning only when needed.